DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ABES INSTITUTE OF TECHNOLOGY, GHAZIABAD

Data Structure-UNIT-2 (QUEUE-Introduction)

BY: AVINASH KUMAR SHARMA

ASSISTANT PROFESSOR, DEPARTMENT OF CSE, ABESIT, GHAZIABAD

Queues

Objectives:

- Describe a queue
- Representation of queue in computer memory
- Implementation of various operation on queues
- Describe priority queue
- Representation of priority queue in computer memory
- Application of queues

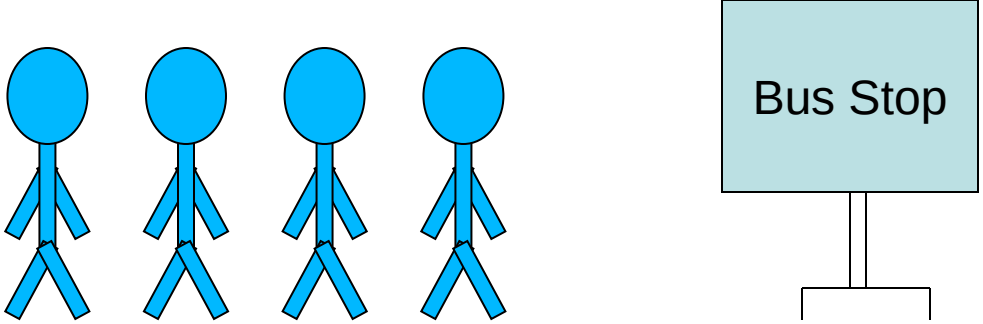
Introduction

A queue is a line of persons waiting for their turn at some service counter.

Service counter can be a ticketing window of a cinema hall or ticketing window of railway station etc.

Depending on the type of service provided by the service counter and number of person interested in service, there could be queue of varying lengths.

The service at the service counter is provide on the first come first serve (FCFS) basis, i.e., in order of their arrival in the queue.



Suppose that at service counter, t1 units time is needed to provide a service to a single person and on average a new person arrive at the counter in t2 units of time. The following possibilities may arise:

1. If t1<t2, then the service counter will be free for some time before a new person arrives at the service counter. Hence no queue in this case.
2. If t1>t2, then service counter will be busy for some time even after the arrival of a new person. Therefore this person has to wait for some time. Similarly other persons arriving at the service counter have to wait. Hence a queue will be formed.
3. If t1=t2, then as a person leaves the service counter, a new person arrives at the service counter. Hence no queue in this case also.

Definition

A queue is a linear list in which insertion can take place at one end of the list, called the rear of the list, and deletion can take place at other end called front of the list.

Behavior of the queue is FIFO system.

In queue terminology insertion and deletion operations are known as enqueue and dequeue operations.

Queue which can accommodate maximum of 10 elements

0	1	2	3	4	5	6	7	8	9

An empty queue

0	1	2	3	4	5	6	7	8	9
5	7	12	8	9					

front rear

Queue after enqueuing elements 5,7,12,8,9 in order

0	1	2	3	4	5	6	7	8	9
		12	8	9					

front rear

Queue after dequeuing 5,7 from queue

Operations on queues

The following operations are performed on queues:

- **Create queue(q)—**to create queue as an empty queue.
- **Enque(q,i)—**to insert element I in a q.
- **Dequeue(q)—**to access and remove an element of queue q.
- **Peek(q)—**to access the first element of the queue q without removing it.
- **Isfull(q)—**to check whether the queue q is full.
- **Isempty(q)—**to check whether the queue q is empty.

Representation of queues in memory

- Array representation
- Linked list representation

Array representation

The following are the necessary declaration:

```
#define CAPACITY 10
typedef struct
{
        int front;
        int rear;
        int elements[CAPACITY];
}queue;
queue q;
```

In addition to above decoration we will use the declaration

```
typedef enum {false, true } Boolean;
```

Array representation of linear queue

front	rear
-1	-1

0	1	2	3	4	5	6	7	8	9

Representation of queue in memory

front	rear
0	4

0	1	2	3	4	5	6	7	8	9
5	7	12	8	9					

Representation of queue in memory

front	rear
2	4

0	1	2	3	4	5	6	7	8	9
		12	8	9					

Representation of queue in memory

Problems with linear queue

front	2
rear	9

0	1	2	3	4	5	6	7	8	9
		12	8	9	9	3	10	11	20

Representation of queue in memory

It is not possible to enqueue more elements as such, though two position in the linear queue are vacant.

To overcome this problem, the elements of the queue are moved forward, so that the vacant positions are shifted toward the rear end.

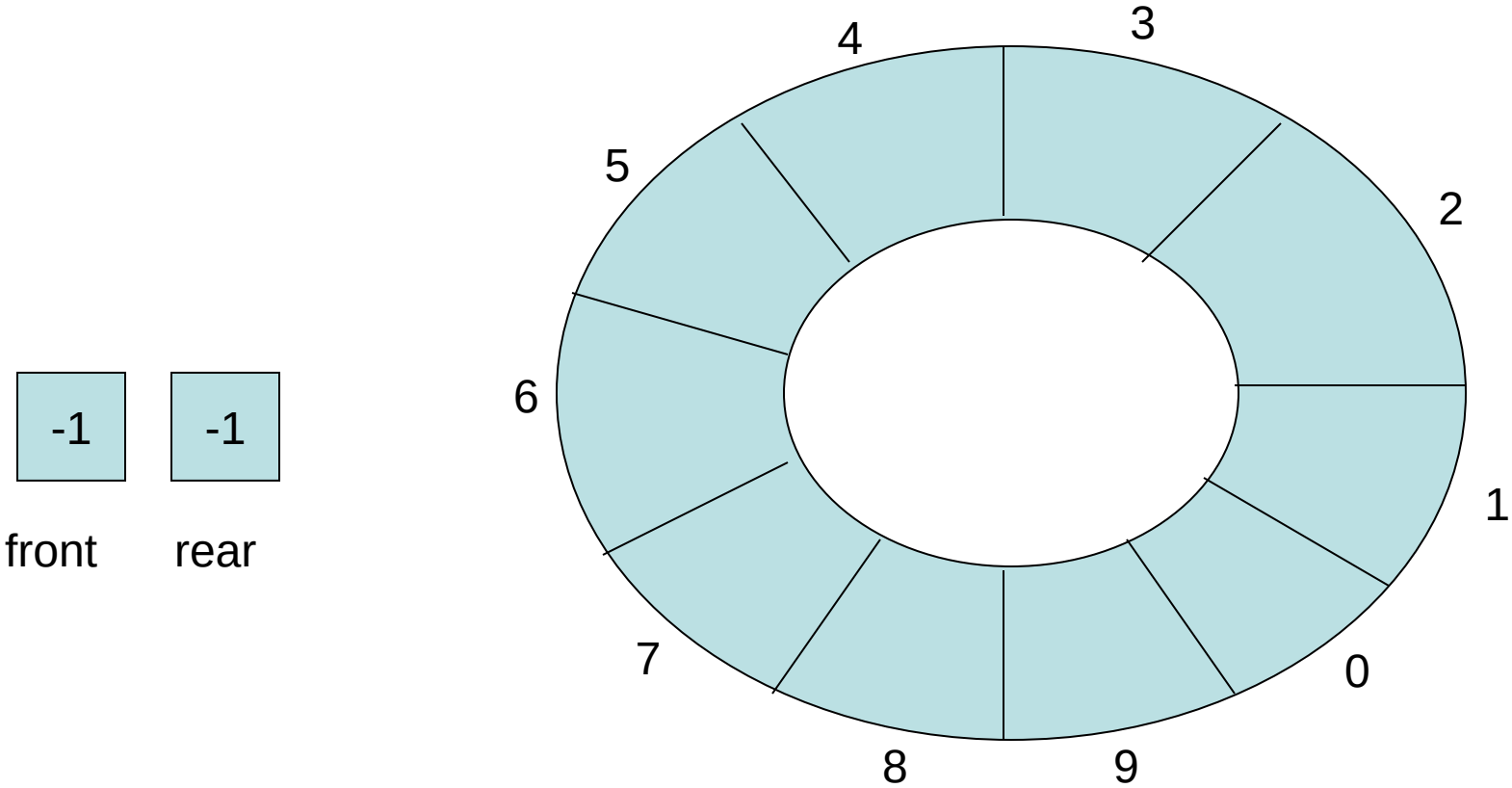
front	0
rear	7

0	1	2	3	4	5	6	7	8	9
12	8	9	9	3	10	11	20		

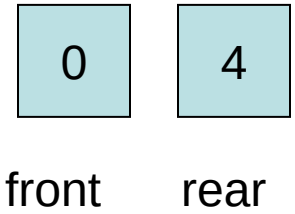
Representation of queue in memory

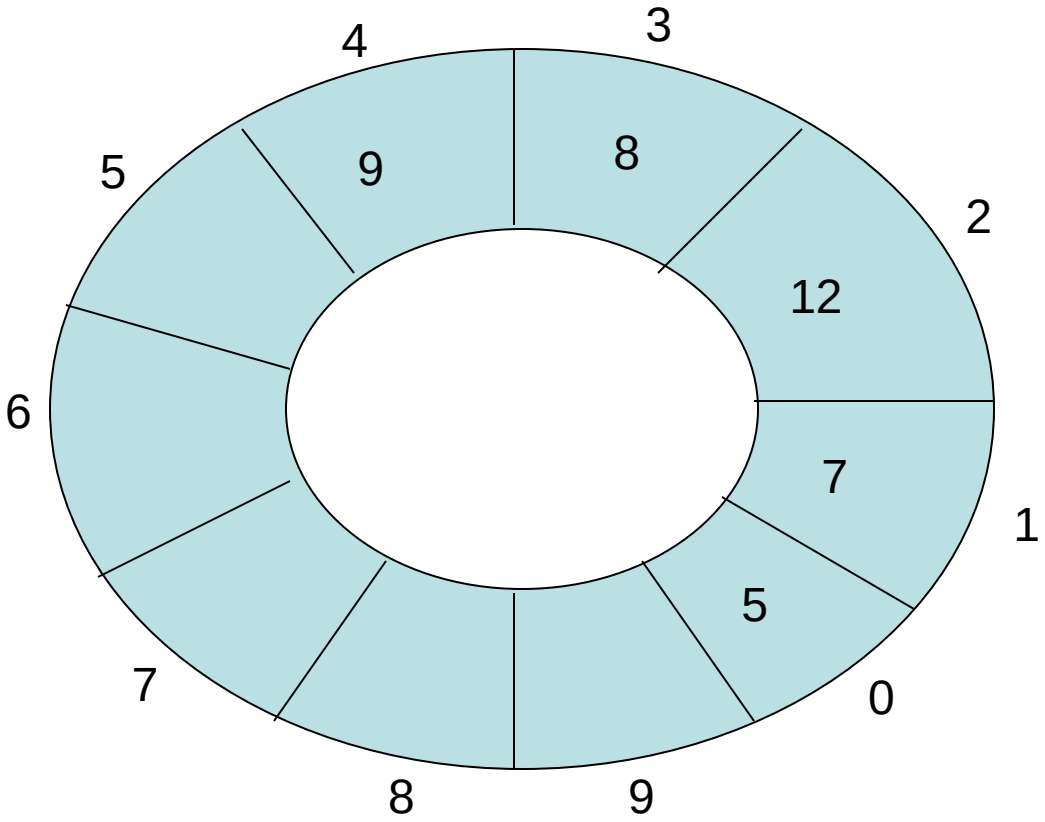
Circular queue

This difficulty of managing front and rear can be overcome if we treat the queue position with index 0 as a position that comes after position with index 9 i.e., we treat the queue as circular.

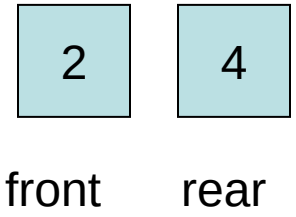


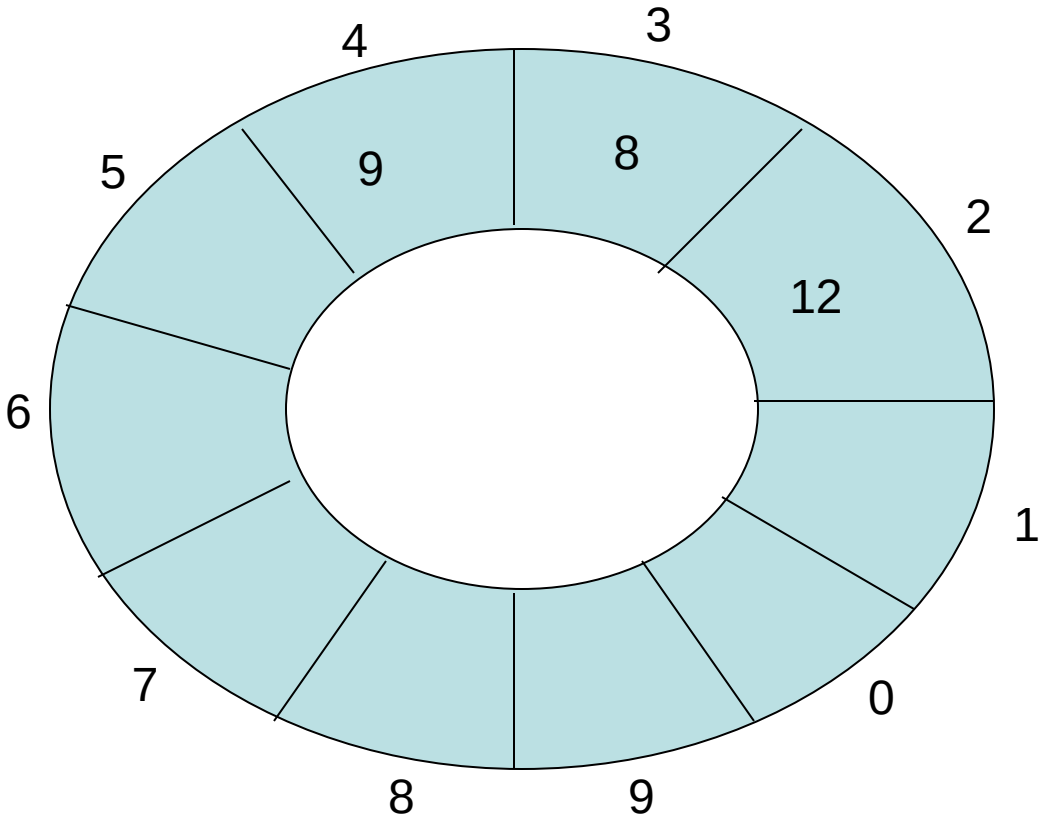
Representation of a queue



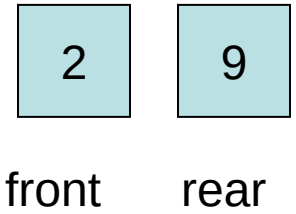


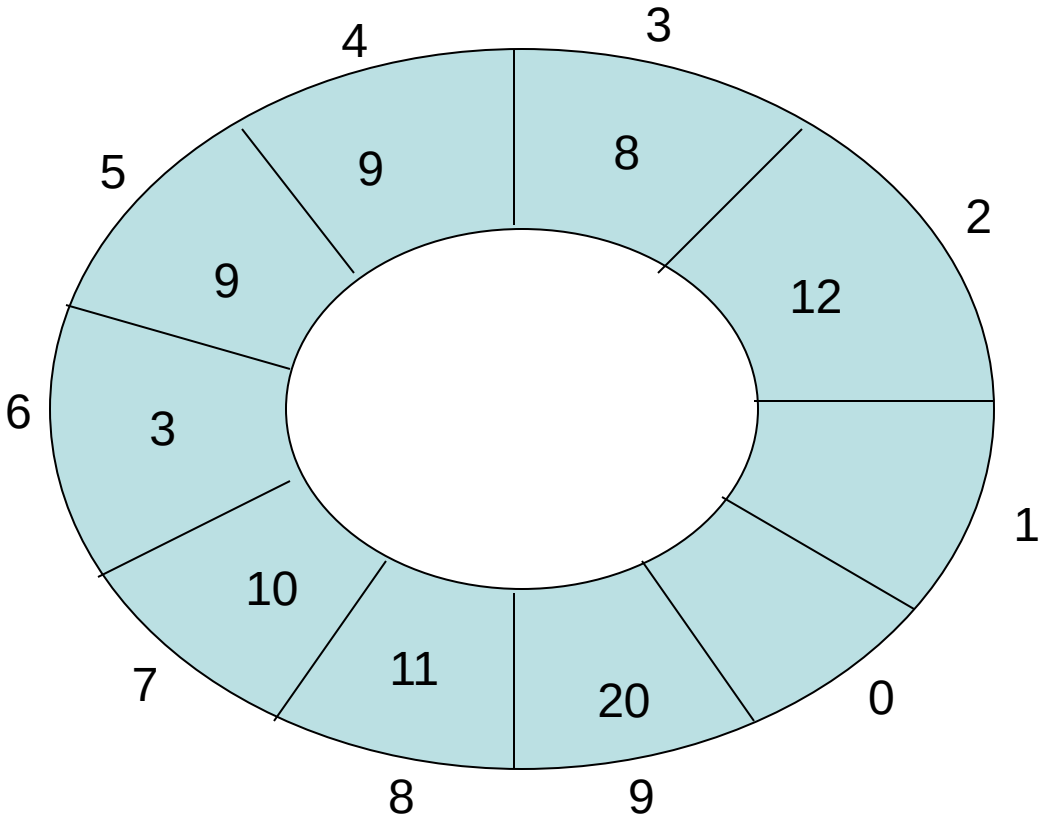
Representation of a queue



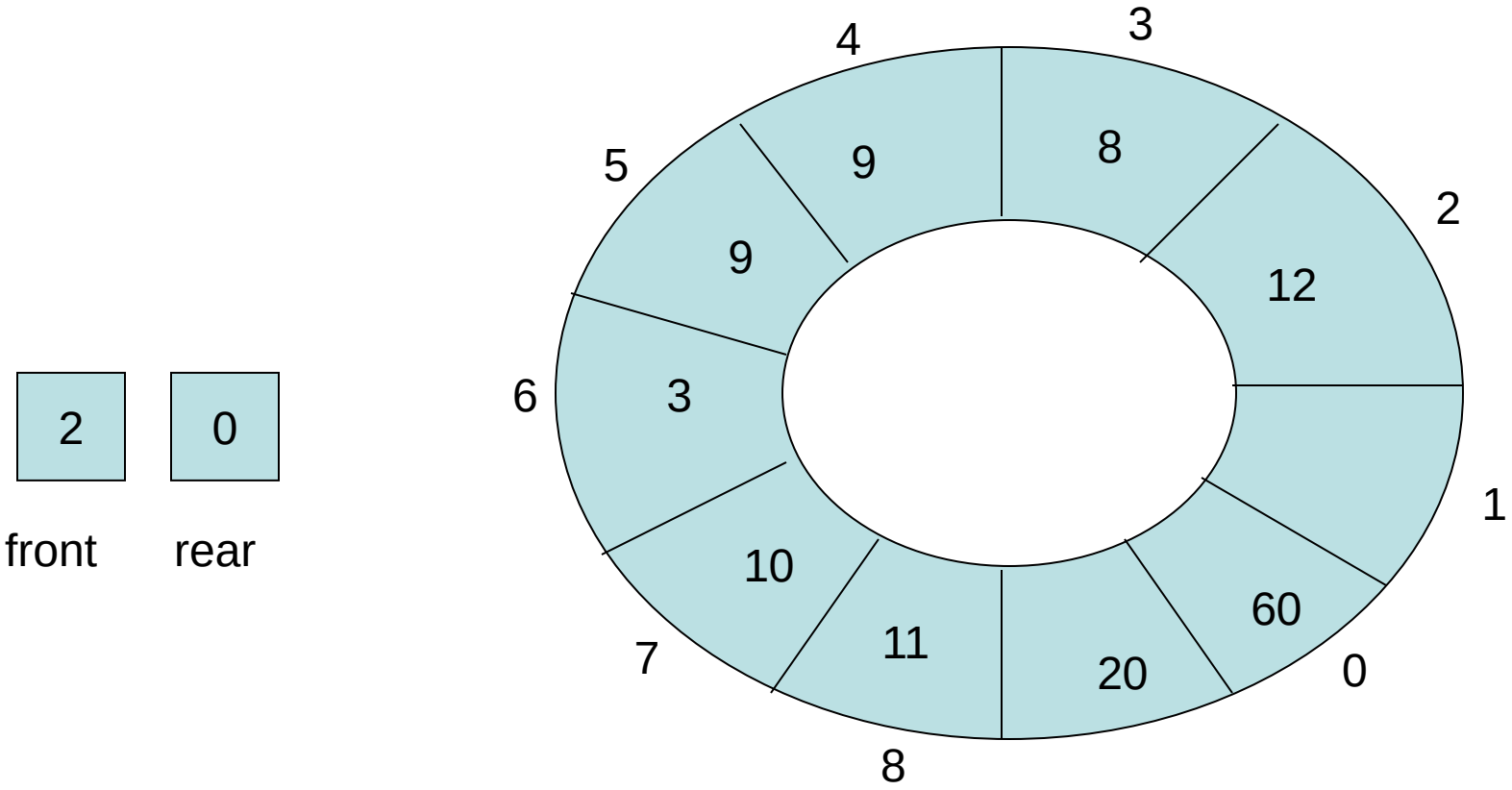


Representation of a queue





Representation of a queue



Implementation of operations on a linear queue

- **Creating an empty queue:**

Before we can use a queue, it must be created.

As the index of array element can take place in between 0 to CAPACITY-1

The purpose of initializing the queue is served by assigning value -1 (sentinel value) to front and rear variables.

```
Void createqueue(queue *pq)
{
   pq->front=pq->rear=-1;
}
```

Testing the queue for underflow

```
Boolean isempty(queue *pq)
{
    if(pq->front==-1)
        return true;
    else
        return false;
}
```

Testing the queue for overflow

```
Boolean isfull(queue *pq)
{
    if((pq->front==0)&&(pq->rear==CAPACITY-1))
        return true;
    else
        return false;
}
```

Enqueue operation on linear queue

There are two conditions, which can occur, even if the queue is not full:

1. If a linear queue is empty, then the value of the front and the rear variable will be NIL (sentinel value), then both front and rear are set to 0.
2. If the linear queue is not empty, then there are further two possibilities:
   1. If the value of the rear variable is less than CAPACITY-1, then the rear variable is incremented.
   2. If the value of the rear variable is equal to CAPACITY-1, then the elements of the linear queue are moved forward, and the front and rear variables are adjusted accordingly.

Enqueue operation on linear queue

```
Void enqueue(queue *pq, int value)
{
    int I;
    if(isempty(pq))
            pq->front=pq->rear=0;
    else if (pq->rear==CAPACITY-1)
                    {
                    for(i=pq->front;i<=pq->rear;i++)
                            pq->elements[i-pq->front]=pq->elements[i];
                            pq->rear=pq->rear-pq->front+1;
                            pq->front=0;
                    }
            else
                    {
                    pq->rear++;
                    }
            pq->elements[pq->rear]=value;
}
```

Dequeue operation on linear queue

There are two possibilities:

1. If there was only one element in the linear queue, then after dequeue operation queue will become empty. This state of linear queue is reflected by setting front and rear variables to -1, the sentinel value.
2. Otherwise the value of front variable is incremented.

Dequeue operation on linear queue

```
Int dequeue(queue *pq)
{
   int temp;
   temp=pq->elements[pq->front];
   if(pq->front==pq->rear)
         pq->front=pq->rear=-1;
   else
        pq->front++;
   return temp;
}
```

Accessing the front element

```
Int peek(queue *pq)
{
   return (pq->elements[pq->front])
}
```

Limitation of Linear Queue

If the last position of the queue is occupied, it is not possible to enqueuing any more elements even though some positions are vacant towards the front positions of the queue.

Implementation of operations on a circular queue

- **Testing a circular queue for overflow:**

Two conditions are:

- **(front=0) and (rear=capacity-1)**
- **front=rear+1**

If any of these two condition is satisfied, it means circular queue is full.

```
Boolean isfull(queue *pq)
{
If(((pq->front==0)&&(pq->rear==CAPACITY-1))||(pq->front==pq->rear+1))
    return true;
else
    return false;
}
```

Enqueue operation on circular queue

There are three conditions, which can occur, even if the queue is not full:

1. If a queue is empty, then the value of the front and the rear variable will be NIL (sentinel value), then both front and rear are set to 0.
2. If the queue is not empty, then the value of the rear will be the index of the last element of the queue, then the rear variable is incremented.
3. If the queue is not full and the value of the rear variable is equal to capacity -1then rear variable is set to 0.

Enqueue operation on circular queue

```
Void enqueue(queue *pq, int value)
{
   /*adjust rear variable*/
   if(pq->front==-1)
         pq->front=pq->rear=0;
   else if (pq->rear==CAPACITY-1)
                pq->rear=0;
           else
                pq->rear++;
   /*store element at new rear */
   pq->elements[pq->rear]=value;
}
```

Dequeue operation on circular queue

There are two possibilities:

1. If there was only one element in the circular queue, then after dequeue operation queue will become empty. This state of circular queue is reflected by setting front and rear variables to -1, the sentinel value.
2. If the value of front variable is equal to CAPACITY-1, then set front variable to 0.

If none of the above conditions hold, then the front variable is incremented.

Dequeue operation on circular queue

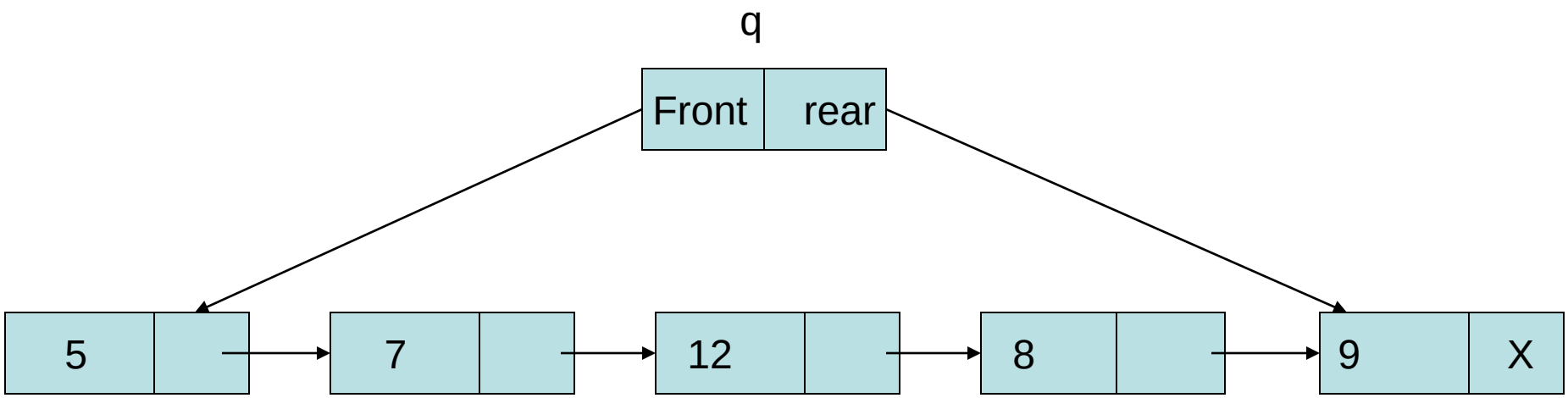
```
Int dequeue(queue *pq)
{
    int temp;
/*store the front element in the temporary variable */
    temp=pq->elements[pq->front];
/*adjust front  variable */
    if(pq->front==pq->rear)
            pq->front=pq->rear=-1;
    else if(pq->front==CAPACITY-1)
               pq->front=0;
            else
               pq->front++;
/*return the dequed element*/
    return temp;
}
```

Representing a queue using a linked list

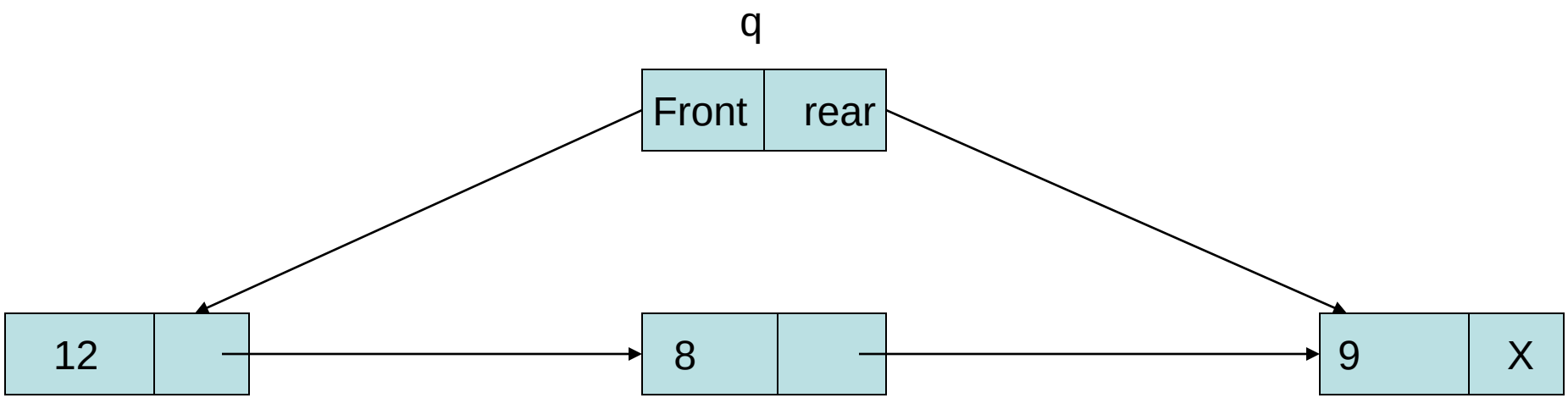
Limitations of array based representation:

- Size of the queue must be known in advance
- We may come across the situation when an attempt to enqueue an element causes overflow.
- However , queue as an abstract data structure can not be full.
- Implementing queue as an array prohibits the growth of queue beyond the finite number of elements.

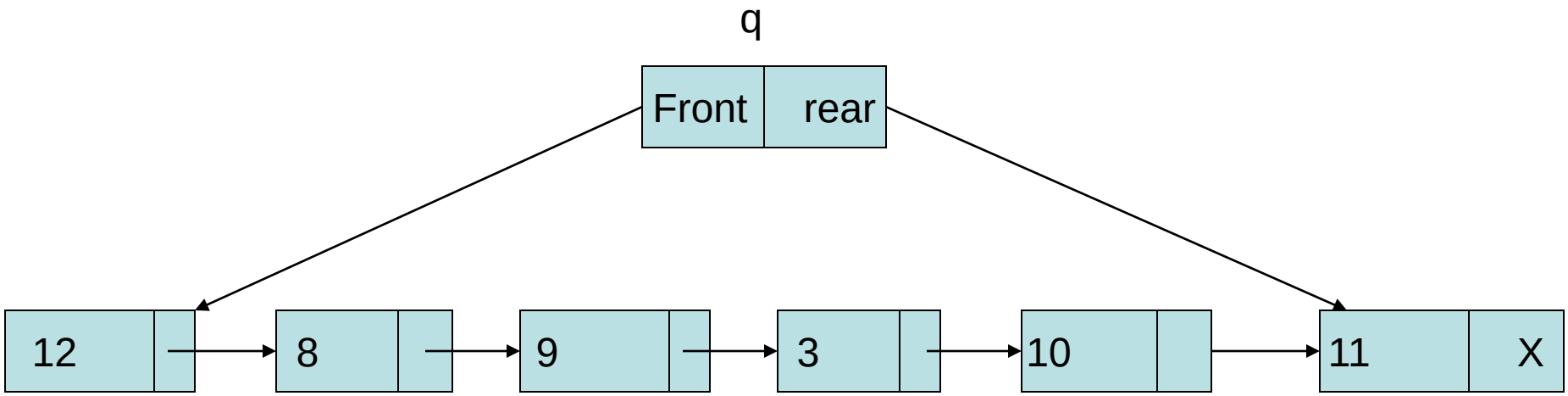
Representation of a queue in memory



Representation of a queue in memory



Representation of a queue in memory



Linked list representation

```
Typedef struct nodetype
{
        int info;
        struct nodetype *next;
}node;

Typedef struct
{
        node *front;
        node *rear;
}queue;
Queue q;
```

Here I have defined two data type named node and queue.

Implementation of operation on a linear queue

- Creating an empty queue:

```
void createqueue(queue *pq)
{
        pq->front=pq->rear=NULL;
}
```

Testing queue for underflow

```
boolean isempty(queue *pq)
{
        if(pq->front==NULL)
                return true;
        else
                return false;
}
```

Testing queue for overflow

Since a queue is represented using a linked list can grow a limit of computer’s memory, therefore overflow condition never occur.

Enqueue operation

```
void enqueue(queue *pq, int value)
{
    node *ptr;
    ptr=(node*)malloc(sizeof(node));
    ptr->info=value;
    ptr->next=NULL;
    if(pq->rear==NULL) /*queue initially empty*/
        pq->front=pq->rear=ptr;
    else
        (pq->rear)->next=ptr;
        pq->rear=ptr;
}
```

Dequeue operation

```
Int dequeue(queue *pq)
{
        int temp;
        node *ptr;
        temp=(pq->front)->info;
        ptr=pq->front;
        if(pq->front==pq->rear) /*only one element*/
                pq->front=pq->rear=NULL;
        else
                pq->front=(pq->front)->next;
        free(ptr);
        return temp;
}
```

Accessing the front element

```
Int peek(queue *pq)
{
   return ((pq->front)->info);
}
```

Disposing a queue

```
Void disposequeue(queue *pq)
{
    node *ptr;
    while(pq->front!=NULL)
    {
        ptr=pq->front;
        pq->front=(pq->front)->next;
        free(ptr);
    }
    pq->rear=(node*)NULL;
}
```

ABES INSTITUTE OF TECHNOLOGY, GHAZIABAD

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

Data Structure-UNIT-2 (Queue)-Lecture-22

BY: AVINASH KUMAR SHARMA

ASSISTANT PROFESSOR, DEPARTMENT OF CSE, ABESIT, GHAZIABAD

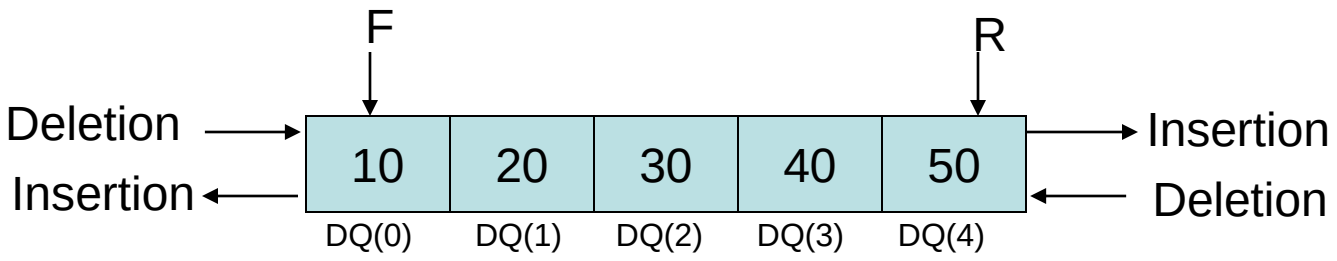
Double Ended Queue (D-Queue)

- It is also homogeneous list of elements in which insertion and deletion operations are performed from both the ends.
- We can insert elements from the rear end or from front end. Hence it is called double ended queue. It is commonly referred as dequeue.

Types of Dqueue

- Types of dqueue are due to the restrictions put to perform either insertion or deletion only at one end
  - Input restricted queue
  - Output restricted queue

Deque


F
R
Deletion
Insertion
10
20
30
40
50
Insertion
Deletion
DQ(0)
DQ(1)
DQ(2)
DQ(3)
DQ(4)

Operations on Deque

- Insertion of an element at the rear end of the queue
- Deletion of an element from the front end of the queue
- Insertion of an element at the front end of the queue
- Deletion of an element from the rear end of the queue

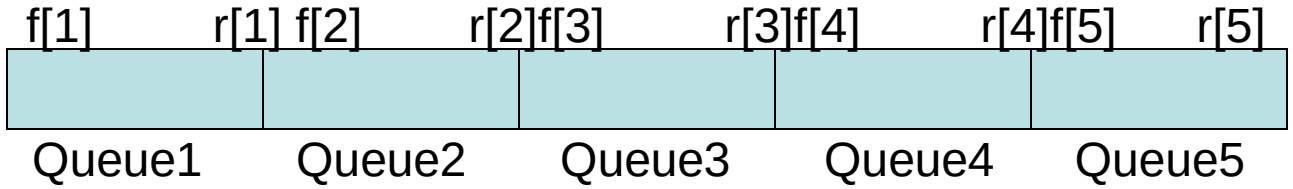
Multiple Queues

Problems with the queues represented with arrays are:

- We have to allocate large space to avoid the overflow.
- Large space will be wasted due to less size of queue.

There exits a trade-off between the number of overflows and the space.

- One possibility to reduce this tradeoff is to represent more than one queue in the same array of sufficient size



Representation of five queues by a single array

Priority Queues

A priority queue is a kind of queue in which each element is assigned a priority and the order in which elements are deleted and processed comes fro the following rules:

- An element of highest priority is processed first before any element of lower priority.
- Two or more elements with the same priority are processed according to the order in which they are added to the queue.

Criteria for determining Priority

- A shortest job is given highest priority over the longer one.
- An important job is given the higher priority over a routine type job
- In commercial computer center, the amount you pay more for the job can determine priority for your job.

Representing a priority Queue in Computer memory

- Using a Linear Linked List
- Using Multiple Queue, one for each priority
- Using Heap

Linear Linked list Representation

```
Typedef struct nodetype
        {
          int info;
          int prn;
          struct nodetype *next;
        }pqueue;
```

Multiple Queue Representation

In this representation, one queue is maintained for each priority number

In order to process an element of the priority queue, element from the first non empty highest priority number queue is accessed.

In order to add a new element to the priority queue, the element is inserted in an appropriate queue for given priority number.

Applications of Queues

- There are several algorithms that use queues to solve problems efficiently.

Eg. We have to use queue for level order traversal in binary tree.

Eg. Performing BFS on graph.

Eg. Most of the simulation related problems.

- When jobs are submitted to a networked printer, they are arranged in order of arrival. Thus , essentially jobs sent to a printer are placed on a queue.
- There is a kind of computer network where disk is attached to one computer, known as file server. User on other computers are given access to files on a first come first serve basis, so the data structure is queue.
- Vertually every real life line a queue. For example, lines at ticket counters at cenema halls, railway stations, bus stands, etc., are queues because the service, i.e. ticket, is provided on first come first serve basis.

Thanks